RationGuard – Database Overview

🗄 Overall Database Structure

- ***Database Used: MongoDB (NoSQL, flexible for government-scale data)***
- ***Purpose: Store all information related to ration beneficiaries, centers, transactions, and detected frauds.***
- ***Design Approach: Modular collections for scalability and quick queries by officials.***
- ***Key Collections:***
  beneficiaries, centers, transactions, alerts, users, ai_logs, reports.

🗂 Beneficiaries Collection

- ***Purpose: Store beneficiary details and ration card information.***
- ***Key Fields:***
  - ***beneficiaryID → Unique system-generated ID***
  - ***cardNumber → Government ration card ID***
  - ***name, familySize, district, centerID***
  - ***eligibility → Category (BPL / APL)***
  - ***lastReceived → Date of last ration collection***
- ***Example Use: Detect duplicate cards or ghost beneficiaries.***

🏢 Centers Collection

- ***Purpose: Store details of distribution centers and responsible officers.***
- ***Key Fields:***
  - ***centerID, district, state, officerName, location***
  - ***totalStock, lastUpdated, status (Active/Under Review)***
- ***Example Use: Identify centers with repeated irregularities.***

💰 Transactions Collection

- ***Purpose: Record every ration transaction for traceability.***
- ***Key Fields:***
  - ***transactionID, centerID, beneficiaryID***

    - ***rationType (Wheat/Rice/Oil/Sugar)***
    - ***quantity, timestamp, status (Normal/Suspicious)***
    - ***flag → true if anomaly detected***
- ***Example Use: Track distribution patterns and find mismatch quantities.***

🚨 *Alerts Collection*

- ***Purpose: Store AI-generated fraud alerts and system flags.***
- ***Key Fields:***
    - ***alertID, centerID, beneficiaryID, type (Duplicate/Quantity Mismatch/etc.)***
    - ***confidenceScore, status (New/Resolved)***
    - ***createdAt, resolvedAt, remarks***
- ***Example Use: Show live alerts on dashboard and record investigation actions.***

👥 *Users Collection*

- ***Purpose: Manage system accounts and access roles.***
- ***Key Fields:***
    - ***userID, name, email, passwordHash, role (Admin/Analyst/Officer)***
    - ***lastLogin, activityLog***
- ***Example Use: Restrict unauthorized access, manage permissions.***

📊 *AI Logs Collection*

- ***Purpose: Store AI processing results for auditing and analytics.***
- ***Key Fields:***
    - ***logID, dataBatchID, modelVersion, accuracy, detectedFrauds, timestamp***
- ***Example Use: Monitor model performance and false positives.***

📈 *Reports Collection*

- ***Purpose: Store generated summaries and exported reports.***
- ***Key Fields:***
    - ***reportID, generatedBy, reportType, timeRange, filePath, createdAt***
- ***Example Use: Fetch past summaries and export data for audits.***

🔗 *Relationships (Conceptual)*

- ***Each Transaction is linked to one Center and one Beneficiary.***
- ***Each Alert references a Transaction and a Center.***
- ***Each User can generate multiple Reports and monitor Centers.***

⚙️ *Additional Features*

- ***Indexing on centerID, beneficiaryID, and timestamp for fast query.***
- ***Data validation with Mongoose schemas.***
- ***Automated backups for reports and alerts.***
- ***Optional integration with ElasticSearch for large-scale fraud analytics.***